SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1709**

Register Number	**P POORNA CHANDU**
Name	**192412199**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 10%

QUESTIONS:2 **Total Marks:20**

Descriptive Test

1. A smart home automation system is being designed to manage lighting, temperature, and security based on user behaviour and environmental conditions. The system must respond to real-time inputs such as motion detection, temperature changes, and time of day while also learning user preferences over time. In this context, explain the different types of intelligent agents (simple reflex, model-based, goal-based, and utility-based agents) and analyze how each type would function within this system. Justify which type of agent or hybrid approach would be most effective for ensuring comfort, energy efficiency, and security.

2. A robot is placed in an unknown maze and must find a path to the exit without prior knowledge of the layout. Explain how uninformed search strategies like Uniform Cost Search (UCS) and Iterative Deepening Search (IDS) can help solve this problem.

   Discuss the advantages and disadvantages of these algorithms in terms of time and space complexity. Relate the problem to different types of intelligent agents and explain how agent design affects decision-making. Suggest which combination of agent type and search strategy would be most effective and justify your choice.

Code of Conduct:

I P.POORNA CHANDU(192412199) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criterion	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Max Marks
Criteria	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**	**Max Marks**
Understanding of Concepts	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions	5
Explanation	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation	5
Application	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis	4
Organization	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers	3
Accuracy	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps	2
Clarity	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand	1

ANSWER 1:

Intelligent Agents in a Smart Home Automation System

An intelligent agent perceives its environment through sensors and acts upon it through actuators. In a smart home system, sensors include motion detectors, temperature sensors, light sensors, cameras, and clocks; actuators include light switches, thermostats, door locks, and alarms. The four classical agent types differ in how they map perception to action.

1. Simple Reflex Agent

How it works: Acts only on the current percept using fixed condition–action rules ("if condition, then action"). It has no memory of past states and no model of the world.

In this system:

- *IF* motion detected *AND* time is night *THEN* turn on hallway light
- *IF* temperature > 28°C *THEN* turn on AC
- *IF* no motion for 10 minutes *THEN* turn off lights

Limitations: It cannot distinguish between an authorized resident and an intruder based on context, cannot adapt to changing conditions it hasn't been explicitly programmed for, and fails when the required information isn't fully observable in the current percept (e.g., "was this room occupied five minutes ago?").

2. Model-Based Reflex Agent

How it works: Maintains an internal state/model of the world, updated with each percept, to handle partial observability. It tracks how the world evolves and how its actions affect it.

In this system:

- Tracks which rooms are currently occupied even when no motion is detected right now (e.g., someone sitting still while reading)
- Maintains a model of "outside temperature trend" to anticipate indoor climate changes
- Remembers whether doors/windows were left open, informing security decisions
- Can infer "user is likely still in the room" even during brief stillness, avoiding false light shut-offs

This agent is more robust than the simple reflex agent because it distinguishes between "no motion right now" and "room is empty," reducing errors like lights turning off while someone is present.

3. Goal-Based Agent

How it works: Uses the internal model plus explicit goals to decide actions — it considers future consequences and chooses actions that will achieve a desired state, not just react to the present.

In this system:

- Goal: "maintain room temperature at 24°C by 6 PM when user typically arrives home" → agent calculates when to start the AC based on current temperature and cooling rate
- Goal: "ensure house is secured before user leaves for work" → agent checks all doors/windows/locks and alerts if unmet
- Goal: "wake user gradually via lighting by 7 AM" → agent plans a sequence of light changes rather than a single reflex action

4. Utility-Based Agent

How it works: Goes further than goal-based agents by using a utility function to measure how "good" a state is, allowing it to choose among multiple ways of achieving a goal — balancing competing objectives like comfort, energy cost, and security.

In this system:

- Weighs *comfort vs. energy efficiency*: instead of just cooling to 24°C, it decides whether to use AC (fast, costly) or open a smart vent (slow, cheap) based on urgency and energy price
- Balances *security vs. convenience*: decides whether unusual motion at 2 AM warrants a silent alert vs. a loud alarm, based on confidence level and risk
- Optimizes across multiple simultaneous factors — e.g., dimming lights slightly to save energy while still meeting a "sufficient brightness" comfort threshold

This is the only agent type capable of genuine trade-off reasoning under competing, sometimes conflicting objectives — which is central to what "smart" home automation actually promises.

Comparative Analysis

Agent Type	Handles Partial Observability	Plans Ahead	Handles Trade-offs	Learns Preferences
Simple Reflex	No	No	No	No
Model-Based	Yes	No	No	No
Goal-Based	Yes	Yes	No (binary goals)	No
Utility-Based	Yes	Yes	Yes	Yes (with learning)

Recommended Approach: A Hybrid, Layered Architecture (Utility-Based as the Core)

No single pure type is sufficient on its own — the most effective real-world design layers them:

1. **Simple reflex layer** for safety-critical, time-sensitive responses that must not wait on reasoning — e.g., smoke detected → sound alarm immediately; door forced open → lock/alert instantly. Speed and reliability matter more than optimization here.
2. **Model-based layer** underlies everything else — the system must maintain a persistent internal model of room occupancy, device states, weather trends, and user location, since sensor data alone (partial observability) is not enough for good decisions.
3. **Goal-based layer** for structured routines — e.g., "secure the house at night," "pre-cool the house before arrival" — where a clear target state exists and a plan of actions is needed to reach it.
4. **Utility-based layer** as the top-level decision-maker, especially for **learning user preferences over time** — since the problem statement explicitly requires this. A utility function can be trained (e.g., via reinforcement learning or weighted preference learning) on inputs like:
    - Energy cost sensitivity
    - Comfort preference (temperature/light tolerance)

    - Security risk tolerance
    - Historical override patterns
5. This lets the system continuously *personalize* trade-offs rather than applying static rules — exactly what distinguishes a "learning" smart home from a simple automated one.

Justification: Comfort, energy efficiency, and security are inherently competing objectives that shift in relative importance depending on context (e.g., security dominates at night when no one is home; comfort dominates when the user is present; energy efficiency dominates when electricity is expensive).

ANSWER 2:

Part 1: Uninformed Search Strategies

Since the robot has no prior knowledge of the maze, it must rely on uninformed (blind) search — algorithms that use only the problem definition (initial state, actions, goal test), with no heuristic guidance.

Uniform Cost Search (UCS)

Expands the node with the lowest cumulative path cost g(n) using a priority queue (essentially Dijkstra's algorithm).

Advantages:

- Complete and optimal — guarantees the cheapest path, even when move costs vary (e.g., rough terrain costs more than a clear path)

Disadvantages:

- Time complexity: O(b^(1+⌊C*/ε⌋))
- Space complexity: same order — must store the entire frontier, which is expensive for a memory-limited robot

Iterative Deepening Search (IDS)

Runs Depth-Limited Search repeatedly with increasing depth limits (0, 1, 2, …) until the goal is found — combining DFS's low memory use with BFS's completeness.

Advantages:

- Time complexity: O(b^d)
- Space complexity: O(bd) — far better than UCS, ideal for a robot with limited onboard memory
- Complete and optimal *for uniform step costs*

Disadvantages:

- Re-explores shallow nodes repeatedly across iterations (small constant overhead)
- Not cost-optimal if move costs are non-uniform — optimizes step count, not actual cost

UCS vs. IDS Comparison

Criterion	**UCS**	**IDS**
Time	O(b^(1+⌊C*/ε⌋))	O(b^d)
Space	High (stores frontier)	Low — O(bd)
Optimal on	Cost	Steps (uniform cost only)
Best for	Variable-cost mazes	Memory-constrained robots

Part 2: Relating to Intelligent Agent Types

Simple Reflex Agent

Reacts only to the current percept (e.g., "wall ahead → turn right"). Cannot perform search at all — no memory, no planning, so no guarantee of reaching the exit.

Model-Based Reflex Agent

Maintains an internal map of visited cells and walls. This is a necessary foundation for search, but on its own it still just reacts — it doesn't plan ahead toward a defined goal.

Goal-Based Agent

Has an explicit goal (reach the exit) and uses search algorithms like UCS/IDS to find a path. This is the natural fit for maze-solving — it plans full action sequences rather than reacting step by step.

Utility-Based Agent

Extends goal-based reasoning by evaluating *which* path is best when multiple objectives exist — e.g., shortest path vs. least energy vs. lowest risk — using a utility function to weigh trade-offs.

Part 3: Recommended Combination and Justification

Utility-based (or at minimum goal-based) agent + IDS as default, switching to UCS when move costs are non-uniform.

- Agent type: Goal-based reasoning is the minimum requirement to run any search meaningfully; utility-based is ideal if the robot must balance multiple concerns (speed, battery, safety) rather than just reaching the exit.
- Search strategy: IDS is preferred by default for its low memory footprint (O(bd)) — critical for a physical robot with no prior knowledge of maze size or depth.
- When to switch: If traversal costs genuinely differ (rough terrain, energy-draining paths), switch to UCS, since IDS only guarantees the fewest *steps*, not the lowest *cost*.

This hybrid mirrors real robotic path-planning systems, which default to memory-efficient iterative methods but switch to cost-based search when energy or risk optimization outweighs raw step count.